

Investigación de Metodologías Ágiles de Desarrollo de Software: XP, Scrum, Kanban y Crystal

Julián Darío Romero Buitrago
Código: 20232020240

Juan Esteban Moreno Durán
Código: 20232020107

Yuber Alejandro Bohorquez Roa
Código: 20231020195

Ingeniería de Sistemas
Universidad Distrital Francisco José de Caldas
Bogotá D.C., Colombia

Resumen—Las metodologías ágiles han transformado la manera en que los equipos de software planifican, desarrollan y entregan productos, priorizando la colaboración, la entrega incremental de valor y la adaptación continua al cambio frente a los modelos predictivos tradicionales. El presente artículo realiza una investigación detallada de cuatro de los marcos de trabajo ágiles más relevantes en la industria y la academia: Extreme Programming (XP), Scrum, Kanban y Crystal. Se describen sus orígenes, principios, roles, artefactos, ciclos de trabajo y prácticas técnicas, y se presenta un análisis comparativo de sus fortalezas, debilidades y contextos de aplicación idóneos. Finalmente, se discuten criterios para la selección de una metodología ágil en función del tamaño del equipo, la criticidad del proyecto y la madurez organizacional, concluyendo que la elección adecuada depende del contexto específico más que de la superioridad intrínseca de un marco sobre otro.

Index Terms—metodologías ágiles, Extreme Programming, Scrum, Kanban, Crystal, desarrollo de software, gestión de proyectos, manifiesto ágil

I. INTRODUCCIÓN

El desarrollo de software ha experimentado una evolución significativa en las últimas décadas en cuanto a los modelos empleados para gestionar proyectos. Los enfoques tradicionales o predictivos, como el modelo en cascada, se caracterizan por una planificación exhaustiva previa a la implementación y por la rigidez frente a cambios en los requisitos [1]. Sin embargo, la creciente volatilidad de los mercados, la incertidumbre en los requisitos del cliente y la necesidad de entregar valor de forma temprana evidenciaron las limitaciones de dichos modelos.

En el año 2001, un grupo de diecisiete profesionales del desarrollo de software se reunió en Utah, Estados Unidos, y redactó el *Manifiesto Ágil*, documento que estableció cuatro valores y doce principios fundamentales orientados a favorecer a los individuos y las interacciones sobre los procesos y las herramientas, el software funcionando sobre la documentación exhaustiva, la colaboración con el cliente sobre la negociación contractual, y la respuesta ante el cambio sobre el seguimiento de un plan [2]. A partir de este manifiesto surgieron o se consolidaron múltiples marcos de trabajo agrupados bajo el paraguas de las metodologías ágiles, entre los cuales destacan Extreme Programming (XP), Scrum, Kanban y Crystal.

El objetivo de este artículo es presentar una investigación detallada y comparativa de estas cuatro metodologías, abordar

do su contexto histórico, fundamentos teóricos, roles, artefactos, prácticas y ciclos de trabajo, con el fin de proporcionar un marco de referencia que apoye la selección de la metodología más adecuada según las características particulares de un proyecto de software.

El resto del documento se organiza de la siguiente manera: la Sección II describe los fundamentos comunes de la agilidad; las Secciones III a VI abordan en detalle XP, Scrum, Kanban y Crystal respectivamente; la Sección VII presenta un análisis comparativo; la Sección VIII discute criterios de selección; y la Sección IX concluye el trabajo.

II. FUNDAMENTOS DE LA AGILIDAD

II-A. El Manifiesto Ágil

El Manifiesto para el Desarrollo Ágil de Software establece los siguientes cuatro valores [2]:

1. Individuos e interacciones sobre procesos y herramientas.
2. Software funcionando sobre documentación extensiva.
3. Colaboración con el cliente sobre negociación contractual.
4. Respuesta ante el cambio sobre seguimiento de un plan.

Estos valores se complementan con doce principios que enfatizan la satisfacción del cliente mediante entregas tempranas y continuas, la aceptación de cambios en los requisitos incluso en etapas avanzadas del proyecto, la entrega frecuente de software funcional, la colaboración diaria entre desarrolladores y personas de negocio, la construcción de proyectos en torno a individuos motivados, la comunicación cara a cara, el software funcionando como principal medida de progreso, el desarrollo sostenible, la atención continua a la excelencia técnica, la simplicidad, los equipos auto-organizados y la reflexión periódica para ajustar el comportamiento del equipo [2].

II-B. Desarrollo iterativo e incremental

Todas las metodologías ágiles comparten, en mayor o menor medida, un enfoque iterativo e incremental, en el cual el producto se construye mediante ciclos cortos que permiten obtener retroalimentación temprana y ajustar el rumbo del proyecto [3]. Esta característica las distingue de los modelos secuenciales y constituye la base sobre la cual cada marco de trabajo particular define sus propias reglas, roles y artefactos.

III. EXTREME PROGRAMMING (XP)

III-A. Origen y contexto

Extreme Programming fue formalizada por Kent Beck a finales de la década de 1990, durante su participación en el proyecto Chrysler Comprehensive Compensation System (C3) [4]. XP se concentra fundamentalmente en las prácticas de ingeniería de software y en la calidad técnica del código, más que en la gestión administrativa del proyecto, razón por la cual suele complementarse con marcos de gestión como Scrum.

III-B. Valores

Beck [4] define cinco valores centrales de XP:

- **Comunicación:** fomentar el diálogo constante entre todos los miembros del equipo y con el cliente.
- **Simplicidad:** diseñar e implementar únicamente lo necesario para satisfacer los requisitos actuales.
- **Retroalimentación:** obtener información temprana sobre el estado del sistema mediante pruebas y entregas frecuentes.
- **Coraje:** tomar decisiones difíciles, como refactorizar código o descartar trabajo que no agrega valor.
- **Respeto:** reconocer el valor del trabajo y las opiniones de cada integrante del equipo.

III-C. Prácticas principales

XP se caracteriza por un conjunto de doce prácticas técnicas de gestión, entre las que se destacan:

- **Programación en pares (*pair programming*):** dos desarrolladores trabajan simultáneamente en la misma estación, uno escribiendo código y el otro revisando en tiempo real.
- **Desarrollo guiado por pruebas (*Test-Driven Development, TDD*):** se escriben las pruebas automatizadas antes que el código de producción, siguiendo el ciclo *red-green-refactor*.
- **Integración continua:** el código se integra al repositorio principal varias veces al día, ejecutando pruebas automatizadas en cada integración.
- **Refactorización continua:** mejora constante de la estructura interna del código sin alterar su comportamiento externo.
- **Propiedad colectiva del código:** cualquier miembro del equipo puede modificar cualquier parte del sistema.
- **Diseño simple:** se implementa la solución más sencilla que cumpla los requisitos vigentes, evitando la sobreingeniería.
- **Estándares de codificación:** el equipo adopta convenciones uniformes para mantener la coherencia del código fuente.
- **Cliente en el sitio (*on-site customer*):** un representante del cliente permanece disponible para el equipo durante todo el desarrollo.
- **Entregas pequeñas y frecuentes:** se libera software funcional en ciclos cortos, típicamente de una a dos semanas.

- **Metáfora del sistema:** se emplea una analogía compartida que facilita la comunicación sobre la arquitectura del sistema.
- **Ritmo sostenible:** se evita el exceso de horas extra para mantener la productividad a largo plazo.
- **Planning game:** el cliente y el equipo negocian conjuntamente el alcance y la prioridad de las historias de usuario en cada iteración.

III-D. Ciclo de vida

El ciclo de XP se organiza en iteraciones cortas (una a dos semanas) enmarcadas dentro de *releases* más amplios. Cada iteración inicia con el *planning game*, continúa con el desarrollo guiado por pruebas y programación en pares, e incluye integración continua durante todo el proceso, finalizando con una entrega funcional revisada por el cliente.

IV. SCRUM

IV-A. Origen y contexto

El término Scrum fue introducido por Takeuchi y Nonaka [5] en un artículo de 1986 sobre desarrollo de nuevos productos, y posteriormente adaptado al desarrollo de software por Ken Schwaber y Jeff Sutherland a mediados de la década de 1990 [6]. Scrum es, ante todo, un marco de trabajo de gestión de proyectos y no prescribe prácticas técnicas de ingeniería, por lo que frecuentemente se combina con prácticas de XP.

IV-B. Roles

De acuerdo con la Guía de Scrum [6], el equipo Scrum está compuesto por tres roles:

- **Product Owner:** responsable de maximizar el valor del producto, gestionando y priorizando el *Product Backlog*.
- **Scrum Master:** facilita la adopción del marco, elimina impedimentos y promueve la mejora continua del equipo.
- **Developers:** profesionales comprometidos con crear cualquier aspecto de un incremento utilizable en cada Sprint.

IV-C. Artefactos

Scrum define tres artefactos principales:

- **Product Backlog:** lista ordenada y dinámica de todo lo que se conoce que es necesario en el producto.
- **Sprint Backlog:** conjunto de elementos del Product Backlog seleccionados para el Sprint, junto con un plan para entregarlos.
- **Incremento:** suma de todos los elementos del Product Backlog completados durante un Sprint, integrados con los incrementos anteriores.

IV-D. Eventos

El marco Scrum organiza el trabajo alrededor de cinco eventos formales [6]:

- **Sprint:** bloque de tiempo fijo, típicamente de una a cuatro semanas, durante el cual se crea un incremento de producto.

- **Sprint Planning:** reunión al inicio del Sprint en la que el equipo define qué se puede entregar y cómo se logrará.
- **Daily Scrum:** reunión diaria de sincronización, limitada a quince minutos, en la que los desarrolladores inspeccionan el progreso hacia el objetivo del Sprint.
- **Sprint Review:** reunión al final del Sprint en la que se inspecciona el incremento y se adapta el Product Backlog.
- **Sprint Retrospective:** reunión de cierre del Sprint destinada a identificar mejoras en procesos, herramientas y relaciones del equipo.

IV-E. Ciclo de trabajo

El ciclo Scrum puede describirse de la siguiente manera: el Product Backlog alimenta el Sprint Planning, del cual surge el Sprint Backlog; durante el Sprint se realizan los Daily Scrums; al finalizar el Sprint se ejecutan el Sprint Review y la Retrospectiva, y el incremento resultante se entrega, reiniciando el ciclo con un nuevo Sprint.

V. KANBAN

V-A. Origen y contexto

Kanban proviene del sistema de producción de Toyota (*Toyota Production System*), desarrollado por Taiichi Ohno en la década de 1940 como parte de la filosofía *lean manufacturing* [7]. Su adaptación al desarrollo de software fue popularizada por David J. Anderson en su obra de 2010 *Kanban: Successful Evolutionary Change for Your Technology Business* [8]. A diferencia de XP y Scrum, Kanban no define iteraciones de duración fija ni roles específicos, sino que se enfoca en la visualización y optimización del flujo de trabajo.

V-B. Principios fundamentales

Anderson [8] propone cuatro principios básicos:

1. Comenzar con lo que se hace actualmente, sin imponer cambios estructurales inmediatos.
2. Acordar perseguir el cambio incremental y evolutivo.
3. Respetar los procesos, roles y responsabilidades actuales.
4. Fomentar el liderazgo en todos los niveles de la organización.

V-C. Prácticas centrales

- **Visualización del flujo de trabajo:** mediante un tablero Kanban dividido en columnas que representan los estados del proceso (por ejemplo: *Por hacer*, *En desarrollo*, *En prueba*, *Hecho*).
- **Límite del trabajo en proceso (Work In Progress, WIP):** se establece un número máximo de tareas permitidas en cada columna, con el fin de evitar la sobrecarga del equipo y reducir el tiempo de ciclo.
- **Gestión del flujo:** se mide y optimiza el tiempo que tardan los elementos en atravesar el sistema, empleando métricas como el *lead time* y el *cycle time*.
- **Políticas explícitas:** las reglas del proceso se documentan y se hacen visibles a todo el equipo.

- **Ciclos de retroalimentación:** se establecen reuniones periódicas de revisión del flujo y de la estrategia, sin ser obligatoriamente de duración fija.
- **Mejora colaborativa mediante experimentación:** los cambios en el proceso se validan empíricamente y se ajustan de forma evolutiva.

V-D. Flujo continuo frente a iteraciones

A diferencia de Scrum, que opera mediante Sprints de duración fija, Kanban se basa en un flujo continuo de trabajo: las tareas se extraen (*pull*) hacia la siguiente etapa del proceso a medida que hay capacidad disponible, en lugar de asignarse en bloques planificados por adelantado [8]. Esta característica hace de Kanban una opción particularmente adecuada para equipos de soporte, mantenimiento o mesas de ayuda, donde la llegada de trabajo es impredecible.

VI. CRYSTAL

VI-A. Origen y contexto

La familia de metodologías Crystal fue desarrollada por Alistair Cockburn, uno de los firmantes originales del Manifiesto Ágil, a partir de investigaciones realizadas para IBM a mediados de la década de 1990 [9]. Cockburn parte de la premisa de que no existe una única metodología óptima para todos los proyectos, sino que el proceso debe adaptarse al tamaño del equipo y a la criticidad del sistema desarrollado.

VI-B. Familia de metodologías según tamaño y criticidad

Crystal se organiza como una familia de metodologías identificadas por colores, que indican el "peso" del proceso en función del número de personas involucradas, mientras que la criticidad del sistema (medida en función del daño potencial ante un fallo: pérdida de comodidad, dinero discrecional, dinero esencial o vida humana) determina el rigor exigido [9]:

- **Crystal Clear:** para equipos de hasta 6 a 8 personas, colocados en un mismo espacio físico, en proyectos de baja criticidad.
- **Crystal Yellow:** para equipos de 10 a 20 personas.
- **Crystal Orange:** para equipos de 20 a 50 personas, típicamente en proyectos de mediana o gran escala con criticidad moderada a alta.
- **Crystal Red:** para equipos de 50 a 100 personas.
- **Crystal Maroon, Diamond y Sapphire:** variantes para proyectos de mayor tamaño y criticidad, incluyendo sistemas cuya falla puede poner en riesgo vidas humanas.

VI-C. Propiedades esenciales

Cockburn [9] identifica siete propiedades que deben priorizarse en cualquier variante de Crystal, siendo las tres primeras consideradas obligatorias:

- **Entregas frecuentes:** el equipo entrega software funcional en intervalos cortos, idealmente no mayores a algunas semanas o meses.
- **Comunicación osmótica:** los miembros del equipo comparten un espacio físico que permite captar información relevante de forma pasiva.

- **Reflexión (mejora continua):** el equipo dedica tiempo periódico a analizar y ajustar su forma de trabajo.
- Seguridad personal, orientada a que los miembros puedan expresar opiniones sin temor a represalias.
- Enfoque en el trabajo prioritario, evitando la dispersión en tareas de bajo valor.
- Fácil acceso a usuarios expertos que resuelvan dudas sobre el dominio del problema.
- Entorno técnico con automatización de pruebas, control de versiones e integración frecuente.

VI-D. Flexibilidad y ligereza

A diferencia de XP y Scrum, Crystal no prescribe herramientas ni prácticas técnicas específicas de manera estricta; en cambio, ofrece un conjunto de principios y propiedades que cada equipo debe adaptar a su propio contexto, lo que la convierte en la más flexible y menos prescriptiva de las cuatro metodologías analizadas [9].

VII. ANÁLISIS COMPARATIVO

VII-A. Comparación general

La Tabla I resume las características principales de las cuatro metodologías estudiadas.

VII-B. Fortalezas y debilidades

La Tabla II presenta un análisis de las principales fortalezas y debilidades identificadas para cada metodología.

Cuadro I
COMPARACIÓN GENERAL DE LAS METODOLOGÍAS ÁGILES ESTUDIADAS

Criterio	XP	Scrum	Kanban	Crystal
Enfoque principal	Prácticas de ingeniería y calidad técnica del código	Gestión y organización del trabajo en equipo	Visualización y optimización del flujo de trabajo	Adaptación del proceso al tamaño y criticidad del proyecto
Estructura temporal	Iteraciones cortas (1-2 semanas)	Sprints de duración fija (1-4 semanas)	Flujo continuo, sin duración fija	Ciclos de entrega flexibles, definidos por el equipo
Roles definidos	Cliente en sitio, programadores (sin roles muy formales)	Product Owner, Scrum Master, Developers	No define roles obligatorios	Varía según variante; roles ligeros y flexibles
Artefactos clave	Historias de usuario, metáfora del sistema	Product Backlog, Sprint Backlog, Incremento	Tablero Kanban, límites WIP	Documentación mínima adaptada al proyecto
Nivel de prescripción técnica	Alto (TDD, integración continua, pair programming)	Bajo (no define prácticas técnicas)	Bajo (se enfoca en el proceso, no en la técnica)	Muy bajo (solo principios generales)
Gestión del cambio	Aceptación continua mediante iteraciones cortas	Cambios gestionados entre Sprints	Cambios incorporados en cualquier momento (flujo continuo)	Cambios adaptados según reflexión periódica del equipo
Tamaño de equipo recomendado	Pequeño (2-12 personas)	Pequeño a mediano (hasta 10-15 personas por equipo)	Variable, sin restricción explícita	Variable según variante (6 a más de 100 personas)
Contexto de aplicación típico	Proyectos con alta incertidumbre técnica que exigen calidad de código	Proyectos de producto con entregas incrementales de valor	Equipos de soporte, mantenimiento o flujo continuo de solicitudes	Proyectos que requieren adaptar el rigor del proceso a su criticidad

Cuadro II
FORTALEZAS Y DEBILIDADES POR METODOLOGÍA

Metod.	Fortalezas	Debilidades
XP	Alta calidad técnica; retroalimentación rápida; reduce defectos	Exige alta disciplina técnica; requiere cliente disponible en sitio
Scrum	Estructura clara; roles bien definidos; mejora la previsibilidad	Puede generar rigidez si los Sprints no se ajustan al ritmo real del trabajo
Kanban	Gran flexibilidad; visualización clara del flujo; reduce cuellos de botella	Menor estructura puede dificultar la planificación a largo plazo
Crystal	Altamente adaptable a distintos contextos y tamaños de equipo	Menor estandarización dificulta su adopción en organizaciones grandes

VII-C. Complementariedad entre metodologías

Es importante señalar que estas metodologías no son necesariamente excluyentes. En la práctica industrial es común encontrar combinaciones como *Scrumban* (integración de Scrum y Kanban) o el uso de prácticas técnicas de XP dentro de un marco de gestión Scrum, dado que XP se enfoca en el *cómo* se construye el software mientras que Scrum y Kanban se enfocan en *cómo se organiza y gestiona* el trabajo del equipo [3].

VIII. CRITERIOS DE SELECCIÓN DE UNA METODOLOGÍA ÁGIL

La selección de una metodología ágil debe considerar, entre otros, los siguientes factores:

- **Tamaño y distribución del equipo:** equipos pequeños y colocados favorecen XP o Crystal Clear; equipos más grandes pueden requerir Scrum con múltiples equipos

coordinados (por ejemplo, mediante marcos de escalado) o variantes más pesadas de Crystal.

- **Criticidad del sistema:** sistemas cuya falla implica riesgos altos (por ejemplo, software médico o financiero) requieren mayor rigor documental y de pruebas, lo cual favorece variantes como Crystal Orange o Red, o la incorporación estricta de prácticas de XP como TDD.
- **Naturaleza del flujo de trabajo:** si el trabajo llega de forma continua e impredecible (soporte, mantenimiento), Kanban resulta más adecuado que un marco basado en iteraciones fijas.
- **Madurez y cultura organizacional:** organizaciones con procesos ya establecidos pueden beneficiarse del enfoque evolutivo de Kanban, mientras que equipos que requieren mayor estructura y disciplina se benefician de Scrum.
- **Disponibilidad del cliente:** XP requiere un compromiso de tiempo considerable por parte del cliente, lo cual no siempre es viable en todos los contextos organizacionales.

IX. CONCLUSIONES

La investigación desarrollada permite concluir que XP, Scrum, Kanban y Crystal, si bien comparten el fundamento común de los valores y principios del Manifiesto Ágil, difieren sustancialmente en su nivel de prescripción, su enfoque (técnico frente a gerencial) y su flexibilidad. XP se distingue por su fuerte énfasis en prácticas de ingeniería que garantizan la calidad del código; Scrum aporta una estructura de roles, eventos y artefactos que facilita la gestión previsible del trabajo en equipos pequeños; Kanban ofrece un modelo de flujo continuo especialmente útil en contextos de alta variabilidad en la llegada de tareas; y Crystal se posiciona como la familia más adaptable, al reconocer explícitamente que el proceso debe ajustarse al tamaño del equipo y a la criticidad del sistema.

No existe una metodología universalmente superior: la elección adecuada depende del contexto particular del proyecto, del equipo y de la organización. En la práctica, muchas organizaciones optan por combinaciones híbridas que capitalizan las fortalezas de varios de estos marcos de trabajo simultáneamente. Como trabajo futuro, se sugiere complementar este análisis teórico con estudios de caso aplicados a proyectos reales, que permitan contrastar empíricamente el desempeño de cada metodología en condiciones específicas.

REFERENCIAS

- [1] I. Sommerville, *Software Engineering*, 10th ed. Boston, MA, USA: Pearson, 2016.
- [2] K. Beck *et al.*, “Manifesto for Agile Software Development,” 2001. [Online]. Available: <https://agilemanifesto.org/>
- [3] C. Larman, *Agile and Iterative Development: A Manager’s Guide*. Boston, MA, USA: Addison-Wesley, 2004.
- [4] K. Beck and C. Andres, *Extreme Programming Explained: Embrace Change*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2004.
- [5] H. Takeuchi and I. Nonaka, “The New New Product Development Game,” *Harvard Business Review*, vol. 64, no. 1, pp. 137–146, 1986.
- [6] K. Schwaber and J. Sutherland, “The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game,” 2020. [Online]. Available: <https://scrumguides.org/>
- [7] T. Ohno, *Toyota Production System: Beyond Large-Scale Production*. Portland, OR, USA: Productivity Press, 1988.
- [8] D. J. Anderson, *Kanban: Successful Evolutionary Change for Your Technology Business*. Sequim, WA, USA: Blue Hole Press, 2010.
- [9] A. Cockburn, *Crystal Clear: A Human-Powered Methodology for Small Teams*. Boston, MA, USA: Addison-Wesley, 2004.